

## SSC0902 – Organização e Arquitetura de Computadores

### Lista de Exercícios – Assembly RISC-V

#### Instruções Gerais

- Todos os programas devem ser escritos em Assembly RISC-V.
- Utilize chamadas de sistema (syscalls) para entrada e saída.
- Organize o código com comentários explicativos.
- Sempre que aplicável, teste os programas com diferentes valores de entrada.

#### Parte 1 – Fundamentos

1. Escreva um programa que imprima uma mensagem de boas-vindas na tela.
2. Leia um número inteiro fornecido pelo usuário e imprima esse mesmo valor.
3. Leia dois números inteiros e imprima o resultado da soma entre eles.
4. Leia um número inteiro e imprima o seu dobro e o seu triplo.
5. Leia um número inteiro e informe se ele é par ou ímpar.
6. Leia dois números inteiros e imprima o maior deles.

#### Parte 2 – Estruturas de Controle

7. Leia um número N e imprima todos os números de 1 até N.
8. Leia um número N e imprima todos os números de N até 1.
9. Leia um número inteiro e imprima sua tabuada de 1 a 10.
10. Leia um número N e calcule a soma de todos os números de 1 até N.
11. Leia um número inteiro e determine se ele é primo.

#### Parte 3 – Manipulação Numérica

13. Leia um número N e imprima os N primeiros termos da sequência de Fibonacci.
14. Leia um número inteiro e informe quantos dígitos ele possui.
15. Leia um número inteiro e imprima seu valor invertido.
16. Leia um número inteiro e calcule a soma de seus dígitos.
17. Verifique se um número é igual ao seu inverso.
18. Leia dois números inteiros e calcule o MDC utilizando o algoritmo de Euclides.

#### Parte 4 – Vetores e Memória

19. Leia N números inteiros e armazene-os em memória.
20. Imprima todos os elementos de um vetor previamente armazenado.
21. Encontre o maior valor em um vetor.
22. Calcule a soma de todos os elementos de um vetor.
23. Leia um valor e verifique se ele está presente no vetor.
24. Ordene um vetor em ordem crescente usando o seguinte algoritmo:

## Algoritmo Bubble

### **variáveis**

inteiro: aux, num[7]:= { 7, 5, 2, 1, 1, 3, 4} ,i, j, MAX;

### **início**

MAX := 7;

**para** i **de** 0 **até** (MAX-1) **faça**

**para** j **de** (MAX-1) **até** (i+1) **passo** -1 **faça**

**se** num[j-1] > num[j] **então**

            aux := num[j-1];

            num[j-1] := num[j];

            num[j] := aux;

**fim\_se;**

**fim\_para;**

**fim\_para;**

**para** i **de** 0 **até** MAX **faça**

    escreva(num[i]);

**fim\_para;**

**fim**

## Parte 5 – Funções e Pilha

25. Implemente uma função que receba dois números e retorne sua soma.
26. Implemente o cálculo de fatorial utilizando função.
27. Implemente a sequência de Fibonacci utilizando funções.
28. Demonstre o salvamento e restauração de registradores usando a pilha.
29. Implemente o cálculo de fatorial de forma recursiva.

## Parte 6 – Funções que manipulam strings

30. Implemente um programa que lê uma string fornecida pelo usuário, inverte e imprime ela invertida.
31. Implemente a função strcat()
32. Implemente a função strcpy()
33. Implemente a função strcmp()

## Observações Finais

- Priorize clareza e organização do código.
- Utilize comentários para explicar decisões importantes.
- Sempre valide as entradas quando necessário.